

SQL Analytics II

Módulo VIII



Temario

- Tracking Analytics
 - Cláusula OVER()
 - Funciones
 - FIRST_VALUE()
 - LAST_VALUE()
 - LAG()
 - LEAD()
 - ROW_NUMBER()
 - RANK()
 - DENSE_RANK()
 - NTILE()

SQL

OVER

OVER:

- La cláusula **OVER** define una ventana o un conjunto de filas definido por el usuario en un conjunto de resultados de la consulta.
- Una función de ventana calcula entonces un valor para cada fila de la ventana.
- Puede utilizar la cláusula **OVER** con funciones para calcular valores agregados tales como medias móviles, agregados acumulados, totales acumulados o N elementos superiores por resultados del grupo.

SQL

OVER

OVER Sintaxis:

[Nombre_Función] **OVER** ([PARTITION BY ...] [ORDER BY ...])

OVER nos permite definir una '*ventana*' de filas o registros sobre los que se aplica una función de agregación, como ser SUM o Funciones de ventana que cubriremos mas adelante.

PARTITION BY nos permitirá definir la '*ventana*' de datos que mirara la función. (Opcional)

ORDER BY se puede agregar dentro de la función de ventana y se utiliza para definir el orden en que la función se ejecuta en los datos. Esto es independiente del orden en que se muestran los resultados generales. (Opcional)

SQL

OVER

OVER Sintaxis:

```
SELECT sod.ProductID,
       sod.UnitPrice,
       AVG(sod.UnitPrice) OVER() UnitPriceAvg
FROM Sales.SalesOrderDetail AS sod
ORDER BY sod ProductID
```

ProductID	UnitPrice	UnitPriceAvg
707	34,99	465,0934
707	34,99	465,0934
707	34,99	465,0934
707	34,99	465,0934
707	34,99	465,0934
707	34,99	465,0934
707	34,99	465,0934
708	34,99	465,0934
708	34,99	465,0934
708	34,99	465,0934
708	34,99	465,0934
708	34,99	465,0934
708	34,99	465,0934
708	34,99	465,0934
708	34,99	465,0934

o (56) | AdventureWorks2008 | 00:00:00 | 121317 rows

En este ejemplo obtenemos el Id de producto, el precio por unidad al momento de ser vendido y el precio promedio histórico de todos los productos alguna vez vendidos.

SQL

OVER

OVER Sintaxis:

```
SELECT sod.ProductID, sod.UnitPrice,
       AVG(sod.UnitPrice) OVER(PARTITION BY ProductID) AS
       UnitPriceAvg
FROM Sales.SalesOrderDetail AS sod
ORDER BY sod UnitPrice
```

En este ejemplo obtenemos el Id de producto, el precio por unidad al momento de ser vendido y el precio promedio histórico de cada producto por separado.

ProductID	UnitPrice	UnitPriceAvg
707	34,99	30,8865
707	34,99	30,8865
707	34,99	30,8865
707	34,99	30,8865
707	34,99	30,8865
708	34,99	30,4495
708	34,99	30,4495
708	34,99	30,4495
708	34,99	30,4495
708	34,99	30,4495
708	34,99	30,4495
708	34,99	30,4495
708	34,99	30,4495
708	34,99	30,4495
708	34,99	30,4495

o (56) | AdventureWorks2008 | 00:00:00 | 121317 rows

SQL

Tracking Analytics

Funciones Analíticas

¿Qué son las funciones Analíticas?

- Las funciones analíticas calculan un valor agregado a partir de un grupo de filas.
- Las funciones analíticas pueden agrupar los datos que se utilizan para calcular el resultado de la función, pero mostrar un valor por cada registro en el conjunto de resultados, sin agrupar.
- Puede usar las funciones analíticas para calcular medias móviles, totales acumulados, porcentajes o resultados de N valores superiores dentro de un grupo.

SQL

Tracking Analytics

Funciones

Función	Significado
FIRST_VALUE()	Devuelve el primer valor de un conjunto ordenado de valores.
LAST_VALUE()	Devuelve el último valor de un conjunto ordenado de valores.
LAG()	Permite comparar el valor de una fila con el de la fila anterior.
LEAD()	Permite comparar el valor de una fila con el de la fila posterior.
ROW_NUMBER()	Devuelve el número secuencial de una fila dentro de una partición de un conjunto de resultados.
RANK()	Devuelve el rango de cada fila en la partición de un conjunto de resultados.
DENSE_RANK()	
NTILE()	Distribuye las filas de una partición ordenada en un número especificado de grupos

FIRST_VALUE():

- Devuelve el valor mínimo del conjunto de registros.
- Ejemplo:

```
SELECT SalesOrderID, SalesOrderDetailID, OrderQty,
FIRST_VALUE(SalesOrderDetailID)
OVER (PARTITION BY SalesOrderID
ORDER BY SalesOrderDetailID) FirstValue
FROM Sales.SalesOrderDetail
WHERE SalesOrderID IN (43670, 43669, 43667, 43663)
ORDER BY SalesOrderID, SalesOrderDetailID, OrderQty
```

	SalesOrderID	SalesOrderD...	OrderQty	FirstValue
1	43663	52	1	52
2	43667	77	3	77
3	43667	78	1	77
4	43667	79	1	77
5	43667	80	1	77
6	43669	110	1	110
7	43670	111	1	111
8	43670	112	2	111
9	43670	113	2	111
10	43670	114	1	111

QL (12.0 RTM) | MAR-Y-MAR\Usuario (56) | AdventureWorks2008 | 00:00:00 | 10 ro

En este ejemplo la consulta nos devuelve el valor mínimo de SalesOrderDetailID para cada SalesOrderID.

LAG():

- Devuelve el valor del registro inmediatamente anterior del conjunto de registros.
- Ejemplo:

```
SELECT SalesOrderID, SalesOrderDetailID, OrderQty,
LAG(SalesOrderDetailID)
OVER (PARTITION BY SalesOrderID
ORDER BY SalesOrderDetailID) FirstValue
FROM Sales.SalesOrderDetail
WHERE SalesOrderID IN (43670, 43669, 43667, 43663)
ORDER BY SalesOrderID, SalesOrderDetailID, OrderQty
```

SalesOrderID	SalesOrderDetailID	OrderQty	FirstValue
43663	52	1	NULL
43667	77	3	NULL
43667	78	1	77
43667	79	1	78
43667	80	1	79
43669	110	1	NULL
43670	111	1	NULL
43670	112	2	111
43670	113	2	112
43670	114	1	113

En este ejemplo la consulta nos permite obtener el SalesOrderID del registro previo siempre que pertenezca a la misma SalesOrderID. En caso de ser único o el primero, nos devuelve NULL.

ROW_NUMBER():

- Asigna números contiguos y únicos desde 1 hasta el último según la ordenación y por cada partición.
- Ejemplo:

```
SELECT SalesOrderID,
ROW_NUMBER() OVER ( ORDER BY SalesOrderID) NumeroFila
FROM Sales.SalesOrderDetail
```

	SalesOrderID	NumeroFila
1	43659	1
2	43659	2
3	43659	3
4	43659	4
5	43659	5
6	43659	6
7	43659	7
8	43659	8
9	43659	9
10	43659	10
11	43659	11
12	43659	12
13	43660	13

En este ejemplo obtenemos todos los SalesOrderIDs presentes en SalesOrderDetail y se le asigna un numero consecutivo.

ROW_NUMBER():

- Le asigna a cada registro un valor consecutivo dentro de la misma partición/ventana.
- Ejemplo:

```
SELECT SalesOrderID,
ROW_NUMBER() OVER ( PARTITION BY SalesOrderID
                     ORDER BY SalesOrderID) NumeroFila
FROM Sales.SalesOrderDetail
```

	SalesOrderID	NumeroFila
1	43659	1
2	43659	2
3	43659	3
4	43659	4
5	43659	5
6	43659	6
7	43659	7
8	43659	8
9	43659	9
10	43659	10
11	43659	11
12	43659	12
13	43660	1
14	43660	2
15	43661	1

En este ejemplo pedimos que el número de fila se reinicie al cambiar el ID de la orden.

RANK():

- Es igual a ROW__NUMBER, pero no asigna números únicos cuando hay dos registros que toman la misma posición.
- Ejemplo:

```
SELECT SalesOrderID, ProductID, OrderQty,
RANK() OVER ( ORDER BY SalesOrderID,
                OrderQty DESC) Rank
FROM Sales.SalesOrderDetail
```

	SalesOrderID	ProductID	OrderQty	Rank
1	43659	709	6	1
2	43659	711	4	2
3	43659	777	3	3
4	43659	714	3	3
5	43659	773	2	5
6	43659	712	2	5
7	43659	776	1	7
8	43659	774	1	7
9	43659	716	1	7
10	43659	778	1	7
11	43659	771	1	7
12	43659	772	1	7
13	43660	762	1	13
14	43660	758	1	13
15	43661	708	5	15
16	43661	712	4	16
17	43661	715	4	16

En este ejemplo la consulta nos devuelve los números de orden, el id de producto y la cantidad de unidades, y crea un ranking en función de la cantidad de productos ordenados. A mismo número de orden y cantidad de productos mismo ranking.

SQL

Tracking Analytics

Funciones Analíticas

RANK():

- Ejemplo:

```
SELECT SalesOrderID, ProductID, OrderQty,
RANK() OVER ( PARTITION BY SalesOrderID
ORDER BY SalesOrderID, OrderQty DESC) Rank
FROM Sales.SalesOrderDetail
```

En este ejemplo la consulta nos devuelve los números de orden, el id de producto y la cantidad de unidades, y crea un ranking en función de la cantidad de productos ordenados por cada número de orden.

El Ranking se reinicia al cambiar el número de orden.

	SalesOrderID	ProductID	OrderQty	Rank
1	43659	709	6	1
2	43659	711	4	2
3	43659	777	3	3
4	43659	714	3	3
5	43659	773	2	5
6	43659	712	2	5
7	43659	776	1	7
8	43659	774	1	7
9	43659	716	1	7
10	43659	778	1	7
11	43659	771	1	7
12	43659	772	1	7
13	43660	762	1	1
14	43660	758	1	1
15	43661	708	5	1
16	43661	712	4	2
17	43661	715	4	2

DENSE_RANK():

- Al igual que RANK(), DENSE_RANK() tampoco asigna números únicos, pero si asigna números consecutivos.
- Ejemplo:

```
SELECT SalesOrderID, ProductID, OrderQty,
DENSE_RANK() OVER ( ORDER BY SalesOrderID,
                        OrderQty DESC) Rank
FROM Sales.SalesOrderDetail
```

	SalesOrderID	ProductID	OrderQty	Rank
1	43659	709	6	1
2	43659	711	4	2
3	43659	777	3	3
4	43659	714	3	3
5	43659	773	2	4
6	43659	712	2	4
7	43659	776	1	5
8	43659	774	1	5
9	43659	716	1	5
10	43659	778	1	5
11	43659	771	1	5
12	43659	772	1	5
13	43660	762	1	6
14	43660	758	1	6
15	43661	708	5	7
16	43661	712	4	8
17	43661	715	4	8

En este ejemplo la consulta nos devuelve los números de orden, el id de producto y la cantidad de unidades, y crea un ranking en función de la cantidad de productos ordenados por cada número de orden. Aquí el Rank asigna números consecutivos,

SQL

Tracking Analytics

Funciones Analíticas

DENSE_RANK():

- Ejemplo:

```
SELECT SalesOrderID, ProductID, OrderQty,
DENSE_RANK() OVER (PARTITION BY SalesOrderID
                    ORDER BY SalesOrderID,
                           OrderQty DESC) Rank
FROM Sales.SalesOrderDetail
```

	SalesOrderID	ProductID	OrderQty	Rank
1	43659	709	6	1
2	43659	711	4	2
3	43659	777	3	3
4	43659	714	3	3
5	43659	773	2	4
6	43659	712	2	4
7	43659	776	1	5
8	43659	774	1	5
9	43659	716	1	5
10	43659	778	1	5
11	43659	771	1	5
12	43659	772	1	5
13	43660	762	1	1
14	43660	758	1	1
15	43661	708	5	1
16	43661	712	4	2
17	43661	715	4	2

En este ejemplo el Ranking se reasigna al cambiar el número de orden.

NTILE():

- Divide las filas ordenadas en la partición en la cantidad especificada de grupos clasificados de igual tamaño en la medida que sea posible y devuelve el grupo en el que se inscribe una fila dada.
- Ejemplo:

```
SELECT Name,
  CONVERT(NVARCHAR(20),SalesYTD,1) AS SalesYTD,
  NTILE(4) OVER(ORDER BY SalesYTD DESC) AS Quartile
FROM Sales.SalesTerritory
```

En este ejemplo la consulta nos devuelve los números de orden, el total de ventas del año hasta la fecha y los agrupa en 4 grupos en función del total de ventas de forma descendente.

Name	SalesYTD	Quartile
Southwest	8,351,296.74	1
Canada	6,917,270.88	1
Northwest	5,767,341.98	1
Central	4,677,108.27	2
France	3,899,045.69	2
Northeast	3,857,163.63	2
United Kingdom	3,514,865.91	3
Southeast	2,851,419.04	3
Germany	2,481,039.18	4
Australia	1,977,474.81	4

Resumen Módulo VII

- Tracking Analytics
 - Cláusula OVER()
 - Funciones
 - FIRST_VALUE()
 - LAST_VALUE()
 - LAG()
 - LEAD()
 - ROW_NUMBER()
 - RANK()
 - DENSE_RANK()
 - NTILE()